

Unit 4: Image Restoration and Compression

Subtopic 3: Image Compression

Course Instructor: Kiran Bagale

Department of Computer Science and Electronics Engineering

Jan, 2026

Outline

- 1 Introduction to Image Compression
- 2 Compression Metrics
- 3 Types of Redundancy in Images
- 4 Coding Redundancy - Huffman Coding
- 5 Interpixel Redundancy - Run Length Coding
- 6 Psychovisual Redundancy - IGS Coding
- 7 Summary and Comparison

What is Image Compression?

Definition

Image compression is the application of data compression techniques to digital images with the goal of reducing the amount of data required to represent an image while maintaining acceptable image quality.

Why Compress Images?

- **Storage:** Images consume significant storage space
- **Transmission:** Faster transmission over networks
- **Bandwidth:** Efficient use of limited bandwidth
- **Cost:** Reduced storage and transmission costs

Example:

- Uncompressed 1920×1080 RGB image: ~ 6.2 MB
- After compression (JPEG): ~ 0.5 -1 MB (85-90% reduction)

Types of Image Compression

Lossless Compression

- Original image can be perfectly reconstructed
- No information loss
- Moderate compression ratios (2:1 to 10:1)
- Used when quality is critical
- Examples: PNG, GIF, TIFF (LZW)

Lossy Compression

- Approximate reconstruction
- Some information lost
- High compression ratios (10:1 to 100:1)
- Exploits human visual limitations
- Examples: JPEG, JPEG2000

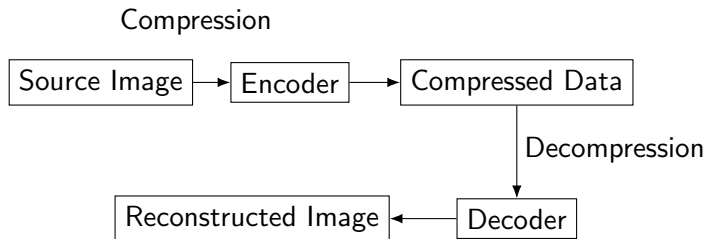
Trade-off

Higher compression → Lower quality (for lossy)

Perfect quality → Lower compression (for lossless)

Image Compression System

General Compression System:



Key Components:

- **Encoder:** Transforms source image to compressed representation
- **Compressed Data:** Reduced size representation
- **Decoder:** Reconstructs image from compressed data

Compression Ratio

Definition

The compression ratio (CR) measures how much the data has been reduced during compression.

Formula:

$$\text{Compression Ratio (CR)} = \frac{n_1}{n_2}$$

where:

- n_1 = Number of information units in original data
- n_2 = Number of information units in compressed data

Interpretation:

- $CR > 1$: Compression achieved
- $CR = 1$: No compression
- $CR < 1$: Data expansion (undesirable)

Compression Ratio - Example

Example 1:

An image of size 512×512 pixels with 8 bits per pixel is compressed to 200,000 bits.

Solution:

$$n_1 = 512 \times 512 \times 8 = 2,097,152 \text{ bits}$$

$$n_2 = 200,000 \text{ bits}$$

$$CR = \frac{2,097,152}{200,000} = 10.49$$

The image is compressed by a factor of 10.49, or approximately 10.5:1.

Space Savings:

$$\text{Space Savings} = \left(1 - \frac{1}{CR}\right) \times 100\% = 90.5\%$$

Relative Data Redundancy

Definition

Relative data redundancy (R_D) quantifies the fractional reduction in data resulting from compression.

Formula:

$$R_D = 1 - \frac{1}{CR} = 1 - \frac{n_2}{n_1}$$

Alternative Expression:

$$CR = \frac{1}{1 - R_D}$$

Properties:

- $0 \leq R_D < 1$
- $R_D = 0$: No redundancy (incompressible data)
- $R_D \rightarrow 1$: High redundancy (highly compressible)
- Expressed as percentage: $R_D \times 100\%$

Relative Data Redundancy - Example

Example:

Using the previous example where $CR = 10.49$:

Solution:

$$R_D = 1 - \frac{1}{10.49} = 1 - 0.0953 = 0.9047$$

This means:

- The original image has 90.47% redundancy
- Only 9.53% of the original data is essential
- 90.47% of the data is redundant and can be removed

Relationship Verification:

$$CR = \frac{1}{1 - R_D} = \frac{1}{1 - 0.9047} = \frac{1}{0.0953} = 10.49\checkmark$$

Average Length of Code

Definition

The average length of code (L_{avg}) is the expected number of bits required to represent each symbol in the compressed representation.

Formula:

$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) \cdot p(r_k)$$

where:

- L = Total number of distinct symbols/gray levels
- $l(r_k)$ = Length (in bits) of code for symbol r_k
- $p(r_k)$ = Probability of symbol r_k

Key Points:

- Lower L_{avg} means better compression
- For fixed-length coding: $L_{avg} = l$ (constant)
- For variable-length coding: $L_{avg} < l$ (if designed well)

Average Length - Example

Example:

Consider a 4-symbol source with the following code:

Symbol	Probability	Code	Length
<i>A</i>	0.5	0	1
<i>B</i>	0.25	10	2
<i>C</i>	0.125	110	3
<i>D</i>	0.125	111	3

Average Length:

$$\begin{aligned}L_{avg} &= 0.5(1) + 0.25(2) + 0.125(3) + 0.125(3) \\&= 0.5 + 0.5 + 0.375 + 0.375 \\&= 1.75 \text{ bits/symbol}\end{aligned}$$

Compare to fixed-length: 2 bits/symbol (need 2 bits for 4 symbols)

Three Types of Redundancy

Data Redundancy in Images

Image data typically contains three types of redundancy that can be exploited for compression:

① Coding Redundancy

- Suboptimal use of code words for pixel values
- Addressed by: Variable-length coding (Huffman)

② Interpixel (Spatial) Redundancy

- Correlation between neighboring pixels
- Addressed by: Run-length encoding, predictive coding

③ Psychovisual Redundancy

- Information not perceived by human visual system
- Addressed by: Quantization, IGS coding

Coding Redundancy

Definition

Coding redundancy exists when the code words assigned to symbols are not optimally matched to the symbol probabilities.

Characteristics:

- Occurs when using fixed-length codes for non-uniform distributions
- More frequent symbols should have shorter codes
- Less frequent symbols can have longer codes

Example:

- Natural images: Gray levels not equally probable
- Some gray levels appear more frequently
- Fixed-length coding assigns same bits to all levels
- Variable-length coding can reduce average length

Solution: Variable-length coding techniques (Huffman, Arithmetic)

Interpixel Redundancy

Definition

Interpixel (spatial) redundancy results from correlations between pixels in an image, particularly between neighboring pixels.

Characteristics:

- Neighboring pixels tend to have similar values
- Smooth regions have high correlation
- Can predict pixel value from neighbors
- Most significant source of redundancy in images

Examples:

- Blue sky: many pixels with similar blue values
- Uniform backgrounds: long runs of same value
- Gradual changes: predictable variations

Solutions: Run-length encoding, Predictive coding, Transform coding

Psychovisual Redundancy

Definition

Psychovisual redundancy refers to information that is not perceptible to the human visual system and can be discarded without significantly affecting perceived image quality.

Characteristics:

- Human eye has limited sensitivity
- Cannot distinguish all intensity levels
- Less sensitive to high-frequency details
- Varies with viewing conditions

Exploitation:

- Quantization: Reduce number of gray levels
- IGS Coding: Improved gray scale coding
- Lossy compression: Discard imperceptible information

Note

Removing psychovisual redundancy leads to lossy compression.

Huffman Coding - Introduction

Definition

Huffman coding is a lossless, variable-length coding technique that assigns shorter codes to more frequent symbols and longer codes to less frequent symbols.

Key Features:

- Optimal prefix-free code
- Based on symbol probabilities
- Greedy algorithm approach
- No code is prefix of another (uniquely decodable)

Properties:

- Lossless compression
- Achieves minimum average code length
- Widely used in image compression (JPEG, PNG)
- Computationally efficient

Huffman Coding - Algorithm

Algorithm Steps:

- ① **List symbols** with their probabilities in descending order
- ② **Combine** the two symbols with lowest probabilities into a single node
 - Sum their probabilities
- ③ **Repeat** step 2 until only one node remains (root)
- ④ **Assign codes** by traversing tree from root to leaves
 - Assign '0' to left branches
 - Assign '1' to right branches
- ⑤ **Record** the code for each symbol

Key Point

The process builds a binary tree (Huffman tree) bottom-up.

Huffman Coding - Example

Example: Encode the following source:

Symbol	Probability	Frequency
a_1	0.4	40%
a_2	0.3	30%
a_3	0.2	20%
a_4	0.1	10%

Step 1: List symbols in order (already sorted)

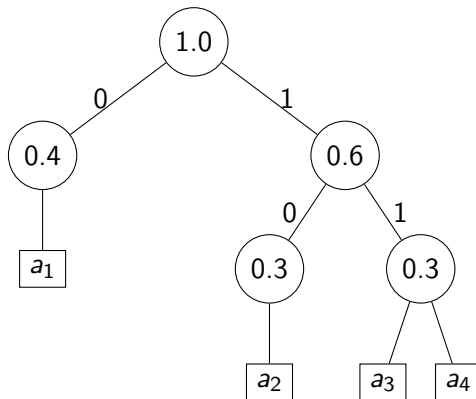
Step 2: Combine a_4 (0.1) and a_3 (0.2) $\rightarrow$ new node (0.3)

Step 3: Combine a_2 (0.3) and previous node (0.3) $\rightarrow$ new node (0.6)

Step 4: Combine a_1 (0.4) and previous node (0.6) $\rightarrow$ root (1.0)

Huffman Coding - Tree Construction

Huffman Tree:



Wait, let me redraw this correctly...

Huffman Coding - Resulting Codes

Reading codes from the tree:

Symbol	Probability	Code	Length
a_1	0.4	0	1
a_2	0.3	10	2
a_3	0.2	110	3
a_4	0.1	111	3

Average Code Length:

$$L_{avg} = 0.4(1) + 0.3(2) + 0.2(3) + 0.1(3) = 1.9 \text{ bits/symbol}$$

Comparison:

- Fixed-length code: 2 bits/symbol (need 2 bits for 4 symbols)
- Huffman code: 1.9 bits/symbol
- Savings: 5% (modest because distribution not heavily skewed)

Huffman Coding - Properties

Advantages:

- Optimal for symbol-by-symbol coding
- Lossless compression
- Fast encoding/decoding
- Prefix-free (uniquely decodable)
- Simple to implement

Disadvantages:

- Requires two passes (count frequencies, then encode)
- Need to transmit/store code table
- Not optimal for correlated symbols
- Efficiency depends on probability distribution

Applications in Image Compression:

- JPEG: Huffman coding of quantized DCT coefficients
- PNG: Can use Huffman coding (or arithmetic coding)
- GIF: Uses LZW (related technique)

Huffman Coding - Image Example

Example: 8-level grayscale image histogram

Gray Level	Probability	Fixed Code	Huffman Code
0	0.35	000	00
1	0.30	001	01
2	0.15	010	100
3	0.10	011	101
4	0.05	100	1100
5	0.03	101	1101
6	0.01	110	1110
7	0.01	111	1111

Average Lengths:

- Fixed: $L_{avg} = 3$ bits/pixel
- Huffman: $L_{avg} = 0.35(2) + 0.30(2) + 0.15(3) + \dots = 2.41$ bits/pixel
- Compression Ratio: $3/2.41 = 1.24$ (or 19.7% savings)

Run-Length Encoding (RLE) - Introduction

Definition

Run-Length Encoding (RLE) is a lossless compression technique that replaces sequences (runs) of identical values with a single value and a count.

Basic Concept:

- Instead of: 5, 5, 5, 5, 5, 5, 5, 5
- Store as: (8, 5) or 8×5
- Represents: "eight occurrences of value 5"

When Effective:

- Images with large uniform regions
- Binary images (documents, line drawings)
- Simple graphics with solid colors
- Medical images (CT, MRI often have uniform regions)

Run-Length Encoding - Format

Representation Format:

Each run is encoded as a pair: (run_length, value)

Example 1: Binary Image Row

Original: 0 0 0 0 0 1 1 1 0 0 0 0 0 0 1 1

Encoded: (5,0) (3,1) (6,0) (2,1)

Example 2: Grayscale Row

Original: 120 120 120 120 85 85 85 85 85 85 200 200

Encoded: (4,120) (6,85) (2,200)

Data Reduction:

- Original: 12 values = 12 bytes (grayscale)
- Encoded: 3 runs = 6 bytes (assuming 1 byte for count + 1 byte for value)
- Compression ratio: $12/6 = 2:1$

Run-Length Encoding - Image Application

2D Image Encoding:

Images are typically processed row-by-row (scanline):

Example: 8×8 Binary Image

```
0 0 0 0 0 0 0 0 0
0 1 1 1 1 1 1 0
0 1 0 0 0 0 1 0
0 1 0 0 0 0 1 0
0 1 0 0 0 0 1 0
0 1 0 0 0 0 1 0
0 1 1 1 1 1 1 0
0 0 0 0 0 0 0 0
```

RLE Encoding:

Row 1: (8, 0)

Row 2: (1, 0)(6, 1)(1, 0)

Row 3: (1, 0)(1, 1)(4, 0)(1, 1)(1, 0)

Row 4: (1, 0)(1, 1)(4, 0)(1, 1)(1, 0)

⋮

Compression Analysis:

- Original: 64 pixels = 64 bits
- After RLE: Varies by image content

Run-Length Encoding - Efficiency Analysis

When RLE Works Well:

- **High compression:** Long runs of identical values
- **Binary images:** Simple black and white documents
- **Computer graphics:** Large uniform color regions
- **Cartoons/logos:** Flat colors, minimal gradients

When RLE Works Poorly:

- **Low compression:** Highly detailed images
- **Natural photographs:** Random pixel variations
- **Noisy images:** Frequent value changes
- **Worst case:** No repeated values (data expansion!)

Example of Data Expansion:

Original: 1 2 3 4 5 6 7 8 (8 values)

Encoded: (1,1) (1,2) (1,3) (1,4) (1,5) (1,6) (1,7) (1,8) (16 values)

Expansion ratio: 2:1 (worse than original!)

Run-Length Encoding - Variants

Common RLE Variants:

- ① **Basic RLE:** (count, value)
 - Simple, works for any data type
- ② **PCX RLE:** Used in PCX image format
 - Single byte run count + value byte
 - Special marker for runs
- ③ **PackBits:** Used in TIFF and TGA
 - Handles both runs and literals efficiently
 - Avoids expansion for non-repeating data
- ④ **CCITT Group 3/4:** For fax transmission
 - Specialized for binary images
 - Variable-length codes for run lengths

Run-Length Encoding - 2D RLE

2D Run-Length Encoding:

Instead of encoding each row independently, exploit vertical redundancy:

Concept:

- Compare current row with previous row
- Encode differences between rows
- More efficient for images with horizontal structures

Applications:

- CCITT Group 4 fax compression
- Used in some TIFF compression schemes
- Video compression (temporal redundancy)

Advantage:

- Better compression for images with horizontal features
- Exploits both horizontal and vertical redundancy

Run-Length Encoding - Properties

Advantages:

- Very simple to implement
- Fast encoding and decoding
- Lossless compression
- Excellent for binary and simple graphics
- Low memory requirements

Disadvantages:

- Poor for complex images
- Can cause data expansion
- Not effective for natural photographs
- Compression ratio varies greatly

Applications:

- Fax transmission (CCITT standards)
- Some video formats (early codecs)
- BMP file format option and Screen captures with large uniform areas

Improved Gray Scale (IGS) Coding - Introduction

Definition

Improved Gray Scale (IGS) coding is a lossy compression technique that reduces the number of bits per pixel while maintaining perceptual image quality through quantization and error diffusion.

Basic Idea:

- Reduce gray levels from 8 bits (256 levels) to 4 bits (16 levels)
- Distribute quantization error to neighboring pixels
- Maintains visual appearance despite reduced precision

Key Concepts:

- **Quantization:** Reduce number of intensity levels
- **Error Diffusion:** Spread quantization error
- **Psychovisual:** Exploits human eye limitations

4-bit Quantization

Quantization Process:

Reduce from 8-bit (0-255) to 4-bit (0-15) representation

Simple Quantization (Truncation):

$$\text{4-bit value} = \left\lfloor \frac{\text{8-bit value}}{16} \right\rfloor$$

or simply: Keep only upper 4 bits

Example:

- 8-bit value: $187 = 10111011_2$
- Simple quantization: $11 = 1011_2$ (upper 4 bits)
- Reconstructed: $11 \times 16 = 176$ (lower 4 bits set to 0)
- Quantization error: $187 - 176 = 11$

Problem

Simple quantization causes visible contouring (false edges)!

Error Diffusion Concept

Problem with Simple Quantization:

- Quantization error is lost
- Creates banding/contouring artifacts
- Degrades visual quality significantly

Solution: Error Diffusion

- Calculate quantization error at each pixel
- Distribute error to neighboring unprocessed pixels
- Preserves average intensity
- Reduces visible artifacts

Mathematical Formulation:

$$e(x, y) = f(x, y) - q(x, y)$$

where:

- $f(x, y)$ = Original pixel value
- $q(x, y)$ = Quantized value
- $e(x, y)$ = Quantization error to diffuse

IGS Coding Algorithm:

- 1: **for** each pixel (x, y) in raster scan order **do**
- 2: Calculate quantized value: $q(x, y) = \text{Quantize}[f(x, y)]$
- 3: Calculate error: $e(x, y) = f(x, y) - q(x, y) \times 16$
- 4: Distribute error to neighbors:
- 5: $f(x + 1, y) = f(x + 1, y) + \frac{7}{16}e(x, y)$
- 6: $f(x - 1, y + 1) = f(x - 1, y + 1) + \frac{3}{16}e(x, y)$
- 7: $f(x, y + 1) = f(x, y + 1) + \frac{5}{16}e(x, y)$
- 8: $f(x + 1, y + 1) = f(x + 1, y + 1) + \frac{1}{16}e(x, y)$
- 9: Store $q(x, y)$ as 4-bit value
- 10: **end for**

Note: This uses Floyd-Steinberg error diffusion weights.

IGS Coding - Error Diffusion Pattern

Floyd-Steinberg Error Diffusion Weights:

Current row		$\times$	
Next row		$\frac{7}{16}$	$\frac{7}{16}$
	$\frac{3}{16}$	$\frac{5}{16}$	$\frac{1}{16}$

Error Distribution:

- $\times$ = Current pixel (already quantized)
- $7/16$ to right neighbor
- $3/16$ to bottom-left neighbor
- $5/16$ to bottom neighbor
- $1/16$ to bottom-right neighbor
- Total: $\frac{7+3+5+1}{16} = 1$ (all error distributed)

IGS Coding - Example

Example: Process one pixel

Input:

- Current pixel value: 187
- Neighbors: 190, 185, 188, 192

Process:

- 1 Quantize: $q = \lfloor 187/16 \rfloor = 11$
- 2 Reconstruct: $11 \times 16 = 176$
- 3 Error: $e = 187 - 176 = 11$
- 4 Distribute error

Result:

- Store 11 (4 bits) for current pixel
- Updated neighbors will be quantized in turn
- Error propagates through image

Error Distribution:

$$\text{Right: } 190 + 11 \times \frac{7}{16} = 194.8$$

$$\text{B-Left: } 185 + 11 \times \frac{3}{16} = 187.1$$

$$\text{Bottom: } 188 + 11 \times \frac{5}{16} = 191.4$$

$$\text{B-Right: } 192 + 11 \times \frac{1}{16} = 192.7$$

Comparison of Quantization Methods:

Simple Quantization	IGS with Error Diffusion
- Visible banding - False contours - Loss of subtle details - Sharp transitions - Posterization effect	- Smooth appearance - No false contours - Preserved details (as noise) - Gradual transitions - Natural appearance

Compression Achieved:

- Original: 8 bits/pixel
- IGS: 4 bits/pixel
- Compression Ratio: 2:1
- Quality: Perceptually acceptable for many applications

Advantages:

- Simple 2:1 compression ratio
- Good visual quality and Fast computation
- No need for complex transforms
- Preserves edges reasonably well

Disadvantages:

- Lossy compression
- Fixed 2:1 compression ratio
- Introduces some noise-like patterns
- Sequential processing (cannot parallelize easily)
- Error accumulation in large uniform areas

Applications:

- Real-time image processing
- Display on limited color devices
- Embedded systems with memory constraints and Print applications (halftoning)

Alternative Error Diffusion Patterns

Other Error Diffusion Kernels:

1. Simple Right-and-Down:

- Right: $1/2$
- Down: $1/2$
- Fastest, but lower quality

2. Stucki Filter:

- Distributes to 12 neighbors
- Better quality, more computation

3. Jarvis-Judice-Ninke:

- Distributes to 12 neighbors
- Different weight distribution

Trade-off:

- More neighbors: Better quality, slower processing
- Fewer neighbors: Faster, may show artifacts

Comparison of Compression Techniques

Technique	Redundancy Type	Compression	Best For
Huffman Coding	Coding redundancy	Lossless, variable (1.2-3:1)	Non-uniform distributions
Run-Length Encoding	Interpixel redundancy	Lossless, variable (up to 100:1)	Binary, simple graphics
IGS Coding	Psychovisual redundancy	Lossy, fixed (2:1)	Grayscale images, memory-limited

Key Observations:

- Each technique addresses different type of redundancy
- Can be combined for better results (e.g., RLE + Huffman)
- Choice depends on image type and requirements
- Modern formats (JPEG, PNG) use combinations of techniques

When to Use Each Technique

Huffman Coding:

- Statistical compression of symbols
- Works with other techniques
- Essential for entropy coding stage

Run-Length Encoding:

- Binary images (fax, documents)
- Computer-generated graphics
- Simple logos and diagrams
- Medical images with uniform regions

IGS Coding:

- When 2:1 compression is sufficient
- Memory-constrained devices
- Real-time requirements
- When lossy compression is acceptable

Combining Compression Techniques

Multi-stage Compression:

Many practical compression systems combine techniques:

Example: JPEG Compression

- 1 Transform (DCT) - removes interpixel redundancy
- 2 Quantization - removes psychovisual redundancy
- 3 Zigzag ordering - groups zeros for RLE
- 4 RLE - encodes zero runs
- 5 Huffman/Arithmetic - removes coding redundancy

Example: PNG Compression

- 1 Prediction - removes interpixel redundancy
- 2 Filtering - optimizes for LZ77
- 3 LZ77 (Deflate) - dictionary-based compression
- 4 Huffman - entropy coding

Summary - Key Metrics

Essential Formulas to Remember:

1. Compression Ratio:

$$CR = \frac{n_1}{n_2} = \frac{\text{Original size}}{\text{Compressed size}}$$

2. Relative Data Redundancy:

$$R_D = 1 - \frac{1}{CR}$$

3. Average Code Length:

$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) \cdot p(r_k)$$

4. Quantization Error (IGS):

$$e(x, y) = f(x, y) - q(x, y) \times \text{scale factor}$$

Summary - Types of Redundancy

Three Main Types:

① Coding Redundancy

- Suboptimal code assignment
- Solution: Huffman, Arithmetic coding
- Lossless

② Interpixel (Spatial) Redundancy

- Correlation between pixels
- Solution: RLE, Predictive, Transform coding
- Can be lossless or lossy

③ Psychovisual Redundancy

- Information imperceptible to human eye
- Solution: Quantization, IGS coding
- Lossy

Summary - Compression Techniques

Main Techniques Covered:

Technique	Type	Typical CR
Huffman Coding	Lossless (Coding)	1.2-3:1
Run-Length Encoding	Lossless (Spatial)	Variable
IGS Coding	Lossy (Psychovisual)	2:1

Key Takeaways:

- No single technique is best for all images
- Modern formats combine multiple techniques
- Lossless: Perfect reconstruction, lower CR
- Lossy: Higher CR, acceptable quality loss
- Understanding redundancy types guides technique selection

Practical Considerations

Choosing a Compression Method:

Consider:

- **Image Type:** Binary, grayscale, color, natural, synthetic
- **Quality Requirements:** Lossless vs. lossy acceptable
- **Compression Ratio:** Required vs. achievable
- **Computational Resources:** Encoding/decoding time, memory
- **Application:** Medical, web, archival, transmission

Examples:

- Medical imaging: Lossless (Huffman + RLE)
- Web photos: Lossy JPEG (multiple techniques)
- Documents/Fax: RLE (CCITT standards)
- Computer graphics: PNG (predictor + Huffman)
- Screen capture: RLE or PNG

Thank You!

Questions?